Full-Text Search

analyse the text once, then search terms not strings

Watch the 4-min video

In one line: full-text search runs every document and every query through the same **analyzer** (tokenize, then normalise: lowercase, fold accents, stem, maybe synonyms), stores the resulting **terms** in an inverted index, and matches terms instead of raw strings, so it is fast, forgiving about word forms, and can rank by relevance.

1 Why LIKE stops working

A shop's search box starts as `SELECT * FROM products WHERE name LIKE '%running shoes%'`. With a hundred products it is fine. With 10 million, it has three separate problems:

- **Speed.** A B-tree index is sorted by the column's first characters, so it can serve `LIKE 'run%'` but not a **leading wildcard**. `'%shoes%'` becomes a full table scan on every keystroke-driven query.
- **Recall.** It compares characters. 'Running Shoes' does not contain 'running shoe', 'Runner's shoe' or 'trainers'. Case is often handled by collation, but plurals, word order and synonyms are not.
- **Ranking.** A row either matches or not. A product whose title is 'Running Shoes' ranks the same as one that mentions shoes once in a 2,000-word description.

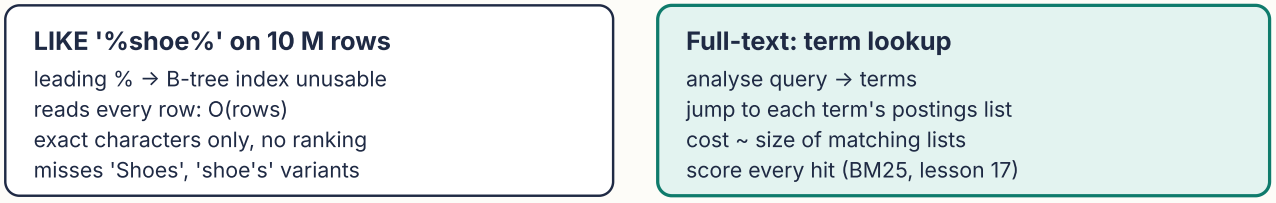


Figure 1. LIKE does work proportional to the table; a full-text engine does work proportional to the matching postings lists.

Databases have partial fixes. PostgreSQL's `pg_trgm` extension builds a GIN index of three-letter fragments that *can* speed up `LIKE '%…%'`, and its built-in full-text search (`to_tsvector('english', name)` plus a GIN index, ranked with `ts_rank`) is enough for many apps. A dedicated engine (Elasticsearch, OpenSearch, Solr, all built on Apache Lucene) adds per-field analyzers, BM25 ranking, facets and horizontal scaling.

2 Analysis: tokenizer plus token filters

An **analyzer** is a pipeline with three stages. **Character filters** clean the raw string (strip HTML, map '&' to 'and'). A **tokenizer** splits it into tokens and records each token's **position**, which phrase queries need later (lesson 15). **Token filters** then change, remove or add tokens. Elasticsearch's default `standard` analyzer is just the Unicode word tokenizer plus lowercase; the built-in `english` analyzer adds a possessive filter ('men's' → 'men'), English stop words and the Porter stemmer.

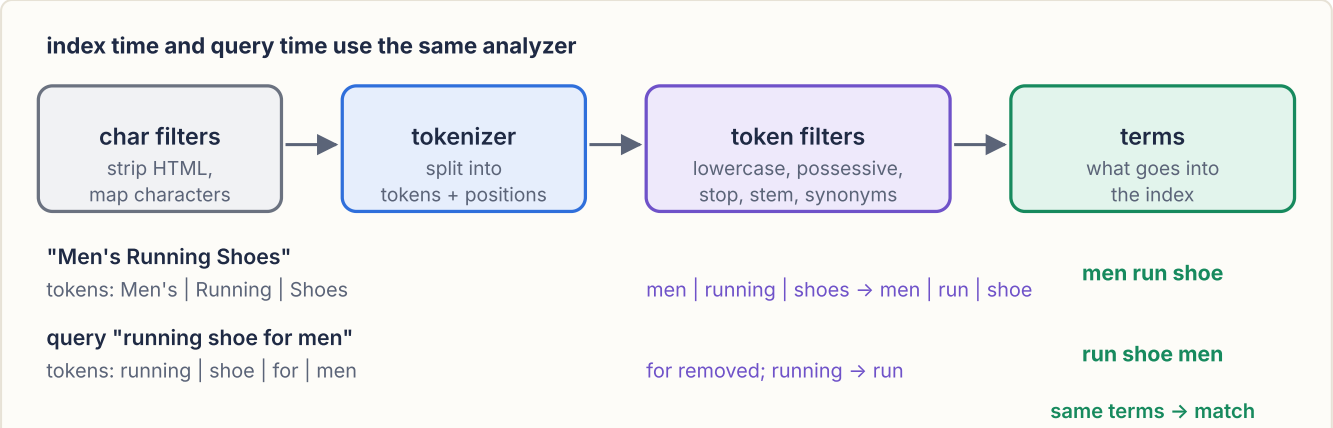


Figure 2. The analyzer runs at index time on the document and at query time on the search text. Only the final terms are compared.

Step	What it does	Example	Watch out for
Lowercase	case-insensitive match	Shoes → shoes	almost always wanted on text fields
ASCII / accent folding	removes diacritics	café → cafe	can merge words that differ in meaning in some languages
Stop words	drops very common words	'shoes for men' → shoes, men	breaks phrases like 'to be or not to be'; BM25 already down-weights them
Stemming (algorithmic)	chops suffixes by rules	running → run, shoes → shoe	Porter leaves 'runner' and 'men' unchanged; can over-merge (university/universe → univers)
Lemmatising / dictionary stemmer	maps to the dictionary form	men → man, ran → run	slower, needs a dictionary (e.g. Hunspell)
Synonyms	adds equivalent terms	sneakers ⇄ trainers	apply at query time so you can change them without reindexing

3 The golden rule: analyse both sides the same way

The index only contains terms. If documents are stemmed but queries are not, the query term 'running' looks for a term that was never stored, because the document's 'running' became 'run'. So each `text` field has one analyzer used at index time and, by default, the same one at search time. The deliberate exceptions are documented ones: **synonyms** are usually added only at search time, and **edge n-grams** only at index time (lesson 21).

Exact values are a different job. Product codes, email addresses, status flags and tags should be `keyword` fields: stored as one un-analysed term, used for filters, sorting and facets (lesson 18). A common pattern is a **multi-field**: `name` as analysed text for search, `name.raw` as keyword for sorting.

4 Forgiving matches: n-grams, fuzziness and synonyms

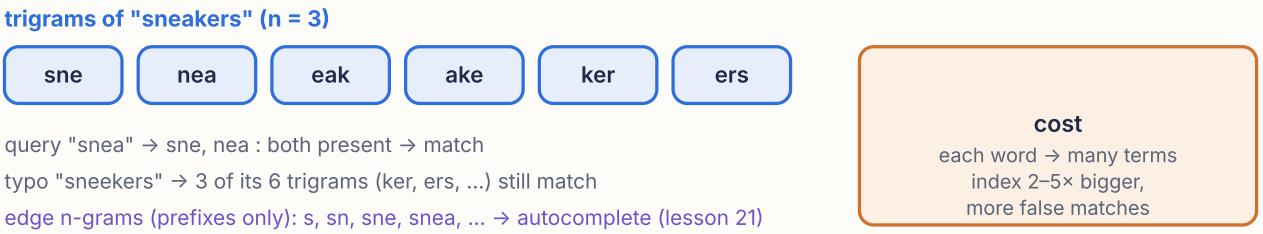


Figure 3. N-grams index fragments so partial words and small typos still share terms. The price is a much bigger index.

- **N-grams** split words into overlapping fragments. They catch partial words and typos but multiply the number of terms, so use them on short fields (names, titles), not on long descriptions.
- **Fuzzy queries** are the cheaper alternative for typos: the engine expands the query term to indexed terms within a small **edit distance** (Levenshtein, max 2 in Lucene). Elasticsearch's `fuzziness: AUTO` allows 0 edits for 1–2 character terms, 1 for 3–5 and 2 for longer ones.
- **Synonyms** fix vocabulary gaps that stemming can't: sneakers/trainers, tv/television, 'nyc'/'new york'. Multi-word synonyms need the graph-aware `synonym_graph` filter at search time.
- **Per-language analyzers.** Stemming rules are language-specific; CJK text has no spaces and needs its own tokenizers (e.g. bigrams or a morphological analyzer like kuromoji for Japanese).

WORKED EXAMPLE: E-COMMERCE PRODUCT SEARCH

Products stay in the database; name and description are copied into a search index whose `name` field uses the `english` analyzer plus a search-time synonym list (`trainers, sneakers, running shoes`). The shopper types **'Running Shoes for Men'**. The query becomes the terms **[run, shoe, men]**: 'for' is a stop word, 'running' and 'shoes' are stemmed. (The video writes 'man'; the Porter stemmer keeps 'men', and you would need a lemmatiser to get 'man'.)

'Men's Runner Shoe' is indexed as [men, runner, shoe], so it matches two of three terms; Porter does not reduce 'runner' to 'run', which is why a `runner → run` synonym or a lemmatiser helps. 'Running Trainers' is indexed as [run, trainer] and matches 'run' directly and 'trainer' through the synonym expansion of 'running shoes'. Both come back in milliseconds, ranked by how many terms matched and where (lesson 17), while a `LIKE` query would have returned neither.

TRADE-OFFS

- **Aggressive stemming** raises recall but lowers precision (Porter turns 'organization' into 'organ').
- **N-grams** catch partial words and typos but bloat the index and add noisy matches.
- **Stop-word removal** saves little space today and breaks phrase queries; many teams keep stop words.
- **A separate engine** gives better search, but it is a second system to keep in sync (lessons 19–20).

COMMON MISTAKES

- Different analyzers at index and query time by accident, so obvious matches never happen.
- Mapping IDs, SKUs or emails as analysed text: 'XB-2017' is split into 'xb' and '2017' and matches too much.
- Changing an analyzer on a live index: existing terms don't change, so you must reindex (lesson 19).
- Never testing the analyzer: use the `_analyze` API to see exactly which terms a string produces.

5 Key terms

Analyzer: char filters + tokenizer + token filters, run on text

Token / term: a unit after tokenizing / after all filters

Stemming: rule-based suffix stripping: running → run

Lemmatisation: dictionary-based base form: men → man

N-gram: overlapping character fragments of a word

keyword field: un-analysed exact value for filters and sorting

6 Interview questions

Q1. Why not use SQL LIKE for search? BASIC

A leading wildcard can't use a B-tree index, so it scans every row; it matches exact characters, missing plurals and variants; and it can't rank results by relevance.

Q2. What does an analyzer do? BASIC

It tokenizes text and normalises the tokens (lowercase, accent folding, stemming, stop words, synonyms), and the same analysis is applied to documents and queries so their terms line up.

Q3. Users search 'iphone15' and find nothing, though 'iPhone 15' exists. What do you change? SCENARIO

Inspect both with the _analyze API; add a word-delimiter or n-gram sub-field (or a synonym) so 'iphone15' and 'iphone 15' produce shared terms, then reindex.

Q4. You need typo tolerance on product names. N-grams or fuzzy queries? SCENARIO

Fuzzy queries first: no index growth, fuzziness AUTO handles 1–2 edits. Use n-grams when you also need partial-word matching, and only on short fields.

Q5. Marketing wants to add synonyms every week. How do you avoid reindexing? SCENARIO

Apply synonyms in a search-time analyzer (synonym_graph) and reload it; index-time synonyms are baked into the terms and need a reindex.

Q6. Why might stemming make results worse? TRICKY

It merges words with different meanings ('universe'/'university' both stem to 'univers'), lowering precision; use a lighter stemmer or keep an unstemmed sub-field and boost exact matches.

Q7. Isn't PostgreSQL full-text search enough? TRICKY

Often yes for one database and modest scale: tsvector + GIN with ts_rank. A dedicated engine wins when you need custom analyzers per field, BM25 tuning, facets at scale or to scale search separately from the database.

Next: SD-P-002 · Inverted Indexes backendroadmap.com